NeoBank 360

Complete Project Explanation — Architecture, Classes, Database & Concepts

NeoBank 360 is a full-stack digital banking application. The **frontend** is an Angular 21 single-page app; the **backend** is a Spring Boot 4 / Java 21 REST API; the **database** is MySQL 8. This document explains every layer, the role of each class type, the database design, and — importantly — **why** each concept is used.

1 · The Big Picture: a Layered Architecture

The project follows the classic layered (n-tier) architecture, where each layer only talks to the one directly beneath it:

```
Angular SPA  →  REST Controller  →  Service  →  Repository  →  MySQL
   (UI)          (HTTP layer)      (logic)     (data access)   (storage)
```

Why this matters: it gives *separation of concerns* — the controller handles only HTTP, the service holds business rules, the repository handles persistence. You can change the database without touching controllers, or change the UI without touching business logic. It also makes the code testable and is the standard enterprise Java pattern.

2 · The Backend, Layer by Layer

2.1 · Entry Point

`BackendApplication.java` is annotated `@SpringBootApplication`. This single annotation bundles `@Configuration`, `@EnableAutoConfiguration` and `@ComponentScan`. It boots an embedded Tomcat server, auto-configures the database connection, and scans the `com.neobank` package to discover all components.

2.2 · Entities — the Data Model in Java

There are 11 entities, each a Java class mapped to a database table using JPA / Hibernate annotations. Example — the `User` entity:

```
@Entity                    // "this class is a DB table"
@Table(name = "users")     // table name
@Data                      // Lombok: getters/setters/toString
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)  // auto-increment PK
    private Long id;

    @Column(nullable = false, unique = true)
    private String email;

    @JsonIgnore                          // never serialize to JSON
    @Column(name = "password_hash")
    private String passwordHash;

    @Enumerated(EnumType.STRING)         // store enum as text
    private Role role;

    @PrePersist void prePersist() { this.createdAt = LocalDateTime.now(); }
}
```

Key concepts and why:

- **@Entity / @Table** — this is ORM (Object-Relational Mapping). Instead of writing SQL you work with Java objects and Hibernate translates them to SQL. Eliminates boilerplate JDBC and reduces SQL-injection risk.
- **@Id + @GeneratedValue(IDENTITY)** — the primary key is auto-generated by MySQL's AUTO_INCREMENT.
- **@JsonIgnore on passwordHash** — security: the hash is never sent to the browser, even by accident.
- **@Enumerated(STRING)** — stores "ADMIN" rather than a fragile integer, so reordering the enum later doesn't corrupt data.
- **@PrePersist** — a JPA lifecycle callback that runs automatically before the first save (here stamping createdAt and defaults).
- **@Data (Lombok)** — generates getters, setters, equals and hashCode at compile time, removing hundreds of lines of boilerplate.

The 11 entities and their tables:

Entity	Table	Represents
User	users	a customer or admin (email, password hash, role, Aadhaar, PAN)
Account	accounts	a bank account (number, type, balance, status, owner)
Transaction	transactions	one money movement on an account (credit / debit / transfer)
LoanProduct	loan_products	an admin-defined loan offering (rate, amount range, tenures)
LoanApplication	loan_applications	a customer's request for a loan
LoanAccount	loan_accounts	an approved, disbursed loan
LoanRepayment	loan_repayments	one EMI instalment in the schedule
Bill	bills	a bill to pay (biller, amount, due date, status)
Budget	budgets	a monthly category spending limit
Reward	rewards	a user's reward-points balance
SystemAuditLog	system_audit_log	a record of every API call

Relationships (JPA associations):

- **@ManyToOne** — many Accounts belong to one User; many Transactions belong to one Account; many LoanApplications belong to one User and one LoanProduct.

- **@OneToOne** — each LoanAccount maps to exactly one LoanApplication (unique = true).
- **fetch = FetchType.LAZY** — the related object isn't loaded until you call its getter. Avoids loading the entire object graph on every query (key for performance).
- **@JoinColumn(name = "user_id")** — defines the foreign-key column.

2.3 · Enums — Controlled Vocabularies

Eight enums constrain values to a fixed set:

Enum	Allowed values
Role	ADMIN, CUSTOMER
AccountType	SAVINGS, CURRENT
AccountStatus	PENDING_APPROVAL, APPROVED, REJECTED
TransactionType	CREDIT, DEBIT, TRANSFER
LoanStatus	PENDING, APPROVED, REJECTED
RepaymentStatus	PENDING, PAID, OVERDUE
BillStatus	PENDING, PAID, OVERDUE
BudgetCategory	GROCERIES, UTILITIES, RENT, ENTERTAINMENT, TRANSFER, OTHER

Why enums: they make illegal states unrepresentable — an account can't be set to status "banana"; the compiler and database both enforce the allowed set. The `AccountStatus.PENDING_APPROVAL` default is a real banking concept: new accounts need admin approval before they can transact.

2.4 · Repositories — Data Access

There are 13 repository interfaces — and that's the interesting part: they're interfaces with no implementation you write.

```
public interface AccountRepository extends JpaRepository<Account, Long> {
    List<Account> findAllByUserId(Long userId);
    Optional<Account> findByAccountNumber(String accountNumber);
    boolean existsByAccountNumber(String accountNumber);

    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("select a from Account a where a.id = :id")
    Optional<Account> findByIdForUpdate(Long id);
}
```

- **extends JpaRepository** — Spring Data JPA generates the implementation at runtime, giving save(), findById(), findAll(), delete() for free. No boilerplate CRUD code.
- **Derived query methods** — findByAccountNumber is parsed from its name into SELECT … WHERE account_number = ?. No SQL written.
- **Optional<Account>** — forces the caller to handle "not found" explicitly instead of risking a NullPointerException.
- **@Lock(PESSIMISTIC_WRITE) + findByIdForUpdate** — pessimistic locking (SELECT … FOR UPDATE), critical for safe money transfers.

2.5 · DTOs — the API Contract

About 40 Data Transfer Objects (e.g. TransferRequest, AccountResponse, LoanApplicationRequestDTO) shape exactly what crosses the network. **Why DTOs instead of exposing entities directly:**

- **Security** — you never leak internal fields; passwordHash lives on User but on no response DTO.
- **Decoupling** — the API contract is independent of the database schema; you can refactor tables without breaking the frontend.

- **Validation** — request DTOs carry validation annotations, so bad input is rejected before reaching business logic.
- **Shaping** — a response can combine fields from several entities (e.g. AdminDashboardDTO aggregates many things into one payload).

2.6 · Services — the Business Logic

The 16 services contain the "what the bank does" logic. The most important pattern is the money transfer in `TransferService`:

```
@Transactional
public TransferResponse transferMoney(TransferRequest request) {
    // 1. validate amount > 0, ownership, both accounts APPROVED & active
    // 2. lock BOTH accounts in a fixed ID order (prevents deadlock)
    Long firstId  = Math.min(sourceId, destId);
    Long secondId = Math.max(sourceId, destId);
    Account a = accountRepository.findByIdForUpdate(firstId)...
    Account b = accountRepository.findByIdForUpdate(secondId)...
    // 3. check sufficient balance
    if (source.getBalance().compareTo(amount) < 0)
        throw new ResponseStatusException(BAD_REQUEST, "Insufficient balance");
    // 4. move money + write TWO transaction ledger rows
    source.setBalance(source.getBalance().subtract(amount));
    destination.setBalance(destination.getBalance().add(amount));
}
```

This single method demonstrates several deep concepts:

- **@Transactional** — wraps everything in one DB transaction. Either both the debit and credit succeed, or both roll back. You can never debit one account without crediting the other. This is ACID atomicity, non-negotiable in banking.
- **Pessimistic locking with ordered acquisition** — both accounts are locked via findByIdForUpdate, always in ascending ID order. If A→B and B→A transfers happen at once, consistent lock ordering prevents a deadlock.
- **BigDecimal for money** — never double/float, which can't represent 0.1 exactly and produce rounding errors. BigDecimal is exact.
- **ResponseStatusException** — cleanly maps a business failure to an HTTP status (400, 403, 404) without custom exception classes.
- **Double-entry bookkeeping** — every transfer writes two Transaction rows (one per account), each recording the running balanceAfter, exactly like a real ledger.

Other notable services:

- **AuthService** — registration & login. Enforces strong password rules (8+ chars, upper, lower, digit, special), checks email/Aadhaar/PAN uniqueness, and hashes the password with BCrypt before saving. Login verifies the hash and issues a JWT.
- **LoanApplicationService** — on admin approval, sets status APPROVED, then orchestrates loanAccountService.createAccountInternal() (computes the EMI) and loanRepaymentService.generateScheduleInternal() (builds the schedule). This is service composition.
- **EmiCalculatorUtil** — the reducing-balance EMI formula (below).

```
// r = annualRate / 12 / 100
emi = (p * r * Math.pow(1+r, n)) / (Math.pow(1+r, n) - 1);
return BigDecimal.valueOf(emi).setScale(2, RoundingMode.HALF_UP);
```

The result is rounded to 2 decimals with HALF_UP. The repayment schedule then makes the final instalment absorb the rounding residual so the loan closes to the exact cent.

2.7 · Controllers — the HTTP Layer

14 controllers expose REST endpoints. Example:

```
@RestController
@RequestMapping("/transfers")
public class TransferController {
    @Autowired private TransferService transferService;

    @PostMapping
    public ResponseEntity<TransferResponse> transferMoney(
            @RequestBody TransferRequest request) {
        TransferResponse r = transferService.transferMoney(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(r);
    }
}
```

- **@RestController** — every method returns data serialized straight to JSON (not an HTML view), because this is a REST API consumed by Angular.
- **@RequestMapping / @PostMapping / @GetMapping** — map URL paths and HTTP verbs to methods, following REST conventions (GET reads, POST creates, PUT updates, DELETE removes).
- **@RequestBody** — Spring deserializes incoming JSON into a DTO automatically.
- **@Autowired** — Dependency Injection: Spring supplies the service instance, so the controller is loosely coupled and testable.
- **ResponseEntity** — sets the precise HTTP status (201 Created for a successful POST).
- **Thin controllers** — they just receive the request and delegate; all logic lives in services.

3 · Security — the Most Important Part of a Banking App

The security package implements stateless JWT authentication with Spring Security.

3.1 · The Flow

- **Register / Login (AuthService)** — password hashed with BCrypt (strength 12). On login the hash is verified and JwtUtil issues a signed token.
- **JwtUtil** — generates and parses JSON Web Tokens, signed with HS256 (HMAC-SHA256). The token carries user id, email and role and expires after 7 days.
- **JwtAuthFilter (extends OncePerRequestFilter)** — runs on every request: reads the Authorization: Bearer header, validates the token, loads the user via CustomUserDetailsService, and populates the SecurityContext with the principal and a ROLE_<role> authority.
- **SecurityConfig** — defines the security filter chain (below).

```
.csrf(disable)                                // token-based API, no cookies
.sessionCreationPolicy(STATELESS)             // no server-side sessions
.requestMatchers("/auth/login","/auth/register").permitAll()
.requestMatchers("/admin/**").hasRole("ADMIN") // admin-only routes
.anyRequest().authenticated()                 // everything else needs login
.addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
```

3.2 · Why Each Choice

- **JWT + stateless** — the server stores no session; each request carries its own proof of identity, so any instance can handle it. Gives horizontal scalability and simplicity.
- **BCrypt** — a deliberately slow, salted, adaptive hash. Even if the database leaks, brute-forcing is expensive and identical passwords produce different hashes.
- **CSRF disabled** — CSRF attacks exploit cookies; auth is via a Bearer header, so CSRF protection is unnecessary and would just block the API.

- **Role-based access control (RBAC)** — /admin/** requires the ADMIN role; @EnableMethodSecurity also allows per-method checks.
- **Custom JSON error responses** — 401 and 403 return clean JSON, not Spring's default HTML, so Angular can handle them gracefully.
- **CorsSecurityConfig** — configures CORS so the browser permits the Angular app (on a different port) to call the API.

4 · Cross-Cutting Concern: AOP Audit Logging

`AuditLogAspect` is the cleverest design choice in the project:

```
@Aspect
@Component
public class AuditLogAspect {
    @Around("execution(* com.neobank.controller..*(..))")
    public Object auditLog(ProceedingJoinPoint joinPoint) throws Throwable {
        long start = System.currentTimeMillis();
        try {
            Object result = joinPoint.proceed();  // run the controller
            return result;
        } finally {
            // record endpoint, method, status, executionTimeMs,
            // acting user  →  system_audit_log
        }
    }
}
```

- **AOP (Aspect-Oriented Programming)** — an "aspect" injects behaviour around methods without modifying them. The pointcut execution(* com.neobank.controller..*(..)) matches every method in every controller.
- **@Around advice** — wraps each controller call, timing it and capturing the response status, the acting user (from the JWT) and any error.
- **Why AOP instead of logging in each controller** — auditing is a cross-cutting concern: it applies everywhere but belongs nowhere in particular. AOP keeps it in one class, so controllers and services stay focused on banking logic. Add a new controller and it's audited automatically. This yields a complete, tamper-evident trail of who called what, when, and how long it took — essential for a regulated banking system.

5 · The Database (MySQL 8)

The schema has the 11 tables matching the entities. Key configuration and design:

- **spring.jpa.hibernate.ddl-auto = validate** — Hibernate validates that the Java entities match the existing schema at startup but does not alter it. In a near-production setup you control the schema explicitly and don't want the ORM silently changing tables.
- **Foreign keys** — enforce referential integrity (an Account must reference a real User).
- **Unique constraints** on email, aadhaar_number, pan_number, account_number — prevent duplicates at the database level (defence in depth, not just app checks).
- **Composite unique constraints** — Budget is unique per (user, category, month); LoanRepayment per (loan account, instalment number).
- **DECIMAL(precision, scale)** for all money columns — the SQL equivalent of BigDecimal, storing exact monetary values.

Domain groupings:

Domain	Tables
Identity	users
Money	accounts, transactions
Lending (a 4-stage pipeline)	loan_products → loan_applications → loan_accounts → loan_repayments
Personal finance	bills, budgets, rewards
Operations	system_audit_log

6 · The Frontend (Angular 21)

A standalone-component single-page application, structured by feature: auth/, dashboard/, loans/, bills/, budget/, rewards/, insights/, admin/, plus shared core/, services/, guards/, interceptors/.

6.1 · Key Building Blocks

- **Components** (.ts + .html + .css) — each screen (login, dashboard, apply-loan, admin-dashboard…). Standalone components are Angular's modern approach, removing the need for NgModules.
- **Services** (auth.ts, transfer.ts, …) — injectable singletons that call the backend via HttpClient and hold shared state, mirroring the backend's service layer.
- **Models** (core/models/*.model.ts) — TypeScript interfaces matching the backend DTOs, giving type safety across the network boundary.
- **Routing** (app.routes.ts) — maps URLs to components and attaches guards.

6.2 · Guards — Route Protection

```
{ path: 'dashboard', component: DashboardComponent,
  canActivate: [AuthGuard] },
{ path: 'admin/dashboard', component: AdminDashboardComponent,
  canActivate: [AuthGuard, AdminGuard] },
{ path: 'login', component: LoginComponent,
  canActivate: [GuestGuard] },
```

- **AuthGuard** — blocks unauthenticated users (redirects to login); also redirects admins away from customer pages.
- **AdminGuard** — restricts admin routes to admin users.
- **GuestGuard** — keeps already-logged-in users off the login/register pages.

Why: the frontend enforces access before navigation, complementing the backend's role checks — defence in depth (both client and server enforce the rules).

6.3 · JWT Interceptor — Automatic Auth

```
@Injectable()
export class JwtInterceptor implements HttpInterceptor {
  intercept(req, next) {
    const token = this.authService.getToken();
    const cloned = req.clone({
      setHeaders: { Authorization: `Bearer ${token}` } });
    return next.handle(cloned)
      .pipe(catchError(err => this.handleAuthError(err)));
  }
}
```

Why an interceptor: instead of manually attaching the token to every HTTP call, the interceptor automatically adds the Authorization header to all requests (skipping login/register) and centrally handles 401 errors (e.g.

logging the user out on token expiry). It is the frontend analogue of the backend's AOP aspect — a single place for a cross-cutting concern.

7 · Summary — Concepts Used and Why

Concept	Where	Why it's used
Layered architecture	whole backend	Separation of concerns, testability
ORM / JPA / Hibernate	entities, repositories	Work with objects, not SQL; avoid injection
Spring Data repositories	repository/	Free CRUD + derived queries, no boilerplate
Dependency Injection	everywhere (@Autowired)	Loose coupling, testable code
DTO pattern	dto/	Secure, decoupled, validated API contract
@Transactional	transfers, loan approval	ACID atomicity — money never half-moved
Pessimistic locking (ordered)	TransferService	Safe concurrent transfers, deadlock-free
BigDecimal + HALF_UP	all money math	Exact monetary arithmetic
Enums	enums/	Make illegal states impossible
JWT (HS256) + stateless	security/	Scalable, sessionless authentication
BCrypt	AuthService	Slow, salted password hashing
RBAC	SecurityConfig, guards	Customers vs admins, enforced both ends
AOP (@Around)	AuditLogAspect	Centralised audit logging, zero per-controller code
REST conventions	controllers	Predictable, standard HTTP API
Angular guards	guards/	Client-side route protection
HTTP interceptor	jwt-interceptor	Auto-attach token, central error handling
TypeScript models	core/models/	Type safety across the wire

The through-line of the whole project is **integrity and security**: every design decision — BigDecimal, @Transactional, pessimistic locks, BCrypt, stateless JWT, role guards on both ends, and AOP auditing — exists because this is software that moves real money and must never lose, duplicate, or expose it.